A PROJECT REPORT ON

COVID-19 DATA VISUALIZATION

(Data Analytics & Business Intelligence)

Project Domain	Data Analytics & Business Intelligence
Project Type	Data Visualization and Interactive Dashboard
Department	Computer Science and Engineering (Artificial Intelligence and Machine Learning)
Primary Objective	Analysis and visual representation of COVID-19 data

Submitted in partial fulfilment of academic project requirements

Department of Computer Science and Engineering

Artificial Intelligence and Machine Learning

Table of Contents

Abstract

The COVID-19 Data Visualization project presents a data analytics and interactive dashboard solution for converting structured COVID-19 records into clear and understandable visual information. During the COVID-19 pandemic, large volumes of information were generated across dates, locations and health indicators. Raw tabular records can be difficult to interpret directly, particularly when users need to understand changes over time or compare locations. This project addresses that problem by providing a structured workflow for loading, validating, cleaning, transforming and visualizing COVID-19 data.

The implementation is designed around a Python-based data processing pipeline and an interactive dashboard. The dataset is loaded from a CSV file using Pandas. The application standardizes column names, identifies relevant fields such as date, location, confirmed cases, deaths, recoveries and active cases, converts dates and numeric values into suitable formats, removes duplicate records and prepares the data for analysis. Date attributes such as year and month can be derived when a valid date field is available. The project then calculates summary Key Performance Indicators (KPIs) and presents them through an interactive dashboard.

The visualization layer uses Plotly charts and Streamlit components. Users can apply date-range and location filters where the corresponding fields are available. The dashboard provides KPI cards for confirmed cases, deaths, recoveries and active cases, together with trend visualizations and a location-based comparison of confirmed cases. A processed dataset preview and a cleaned CSV download option are also provided. The source code separates reusable data-processing functions from dashboard presentation logic, improving readability and maintainability.

The project is developed under the Computer Science and Engineering (Artificial Intelligence and Machine Learning) department and provides a practical foundation for future AIML extensions. The current implementation focuses on descriptive analytics and visualization rather than predictive modeling. Future work can extend the system with forecasting, anomaly detection, automated data updates and machine-learning-based analysis. Overall, the project demonstrates how a structured data pipeline and interactive visualization interface can transform COVID-19 records into a useful analytical view for understanding trends and comparisons.

Keywords: COVID-19, Data Visualization, Data Analytics, Python, Pandas, Plotly, Streamlit, Dashboard, Artificial Intelligence and Machine Learning

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
AIML	Artificial Intelligence and Machine Learning
BI	Business Intelligence
CSV	Comma-Separated Values
ETL	Extract, Transform, Load
KPI	Key Performance Indicator
UI	User Interface
UX	User Experience
API	Application Programming Interface
EDA	Exploratory Data Analysis
ML	Machine Learning

List of Figures

List of Graphs

Chapter 01: INTRODUCTION

1.1 Introduction

Data visualization is the process of presenting data in graphical form so that patterns, comparisons and trends can be understood more easily. In the context of public-health data, visualization is particularly useful because large datasets may contain observations across many dates and locations. A dashboard can provide a consolidated view that is easier to explore than a raw spreadsheet.

The COVID-19 Data Visualization project applies this principle to a structured COVID-19 dataset. The application uses Python-based data processing to prepare the records and then presents the resulting information through an interactive Streamlit dashboard. Pandas is used for tabular data operations, while Plotly is used for interactive charts. The dashboard focuses on descriptive analysis: it summarizes available case indicators and visualizes how those indicators change across time and locations.

The project is intentionally designed as an academic-scale analytics system. It does not claim to provide medical diagnosis, epidemiological prediction or real-time public-health decision support. Instead, it demonstrates a complete data workflow from CSV input to cleaned data, KPI calculation and interactive visualization.

1.2 Problem Statement

- COVID-19 datasets can contain large numbers of records that are difficult to interpret directly in tabular form.
- Comparing confirmed cases, deaths, recoveries and active cases across dates requires repeated manual filtering and aggregation.
- Location-based comparison becomes difficult when the same dataset must be inspected across many regions or countries.
- Raw datasets may contain missing values, duplicate records, inconsistent column names and values stored in unsuitable data types.
- Static tables do not provide an interactive way to explore a selected date range or location.
- Users need a concise dashboard view that combines summary indicators with detailed visual trends.

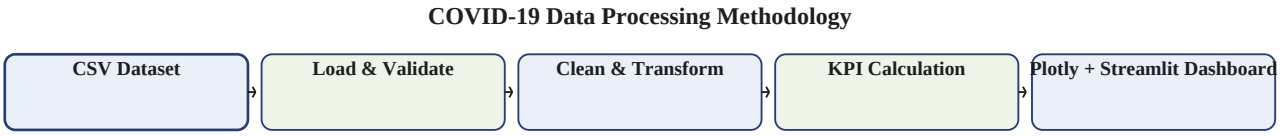
The project addresses these issues through a structured data-processing workflow and an interactive visualization layer. The source code performs validation, cleaning, transformation, KPI calculation and visualization before presenting the results in the dashboard.

1.3 Objectives

- Load COVID-19 data from a CSV file.
- Validate the dataset path and identify available analytical fields.
- Standardize column names for consistent processing.
- Convert dates and numeric fields into suitable formats.
- Remove duplicate records and handle invalid numeric values.
- Create date-derived fields such as year and month when a date field is available.
- Calculate summary KPIs for confirmed cases, deaths, recoveries and active cases when corresponding fields exist.
- Provide date-range and location filters where the dataset supports them.
- Visualize trends using interactive Plotly charts.
- Compare locations using a bar chart of confirmed cases.
- Preview the processed dataset and provide a cleaned-data download.
- Provide a foundation for future AIML-based forecasting and anomaly detection.

1.4 Methodology

The methodology follows a practical data-analytics workflow. First, the CSV dataset is loaded and inspected. Next, column names are standardized so that the application can recognize common variations in field names. Date and numeric columns are converted into appropriate formats, duplicate rows are removed, and invalid numeric entries are handled. After preprocessing, the application applies dashboard filters and calculates KPIs. Finally, Plotly visualizations are rendered inside the Streamlit interface.



1.5 Motivation

The motivation for the project is to demonstrate how raw public-health data can be converted into a form that is easier to explore and interpret. Instead of repeatedly examining CSV rows, a user can view summary indicators and interactive trends in one place. The project also provides a practical application of concepts from data analytics, visualization and Artificial Intelligence and Machine Learning-oriented data processing.

1.6 Organization

This report is organized into five chapters. Chapter 1 introduces the project, problem, objectives and methodology. Chapter 2 presents a concise literature survey and establishes the relevance of data visualization and dashboards. Chapter 3 describes the system development, requirements, technologies, folder structure, data model, pseudocode and application flow. Chapter 4 discusses performance and usability from a project-scale perspective. Chapter 5 concludes the report and presents applications, limitations and future work.

Chapter 02: LITERATURE SURVEY

Data visualization has become an important component of analytical systems because it provides a visual interface between structured data and human interpretation. Dashboards combine multiple indicators and visual elements into a single interface, allowing users to move from an overall summary to more detailed views. In public-health analytics, this approach is useful for understanding changes in reported indicators across time and geography.

Interactive visualization libraries such as Plotly support chart types that are appropriate for time-series and categorical analysis. Line charts are commonly used to communicate changes over time, while bar charts are useful for comparing categories or locations. The selection of chart type should be driven by the analytical question rather than by visual complexity.

Data preparation is another important part of the analytics workflow. Missing values, duplicate records, inconsistent naming and incorrect data types can affect downstream calculations. Therefore, the current project treats validation and cleaning as explicit stages before KPI calculation and visualization. This design is consistent with the broader principle that reliable visual analytics depends on reliable input data.

Dashboard frameworks such as Streamlit make it possible to combine Python-based data processing with interactive user-interface elements. A single application can expose filters, KPI cards, charts and tabular previews without requiring a separate front-end development stack. This makes Streamlit suitable for an academic project where the objective is to demonstrate the complete analytics workflow.

Within an Artificial Intelligence and Machine Learning curriculum, visualization also serves as an important exploratory stage before predictive modeling. Understanding distributions, trends and anomalies can guide the selection of later modeling techniques. The present project stops at descriptive visualization, while future versions can build forecasting and anomaly-detection modules on the prepared data.

Literature-Informed Design Principles

- Data quality should be addressed before analytical calculations.
- Time-series questions are naturally represented with line charts.
- Location or category comparisons are well suited to bar charts.
- Important KPIs should be visible before detailed charts.
- Interactive filters should reduce the effort required to answer focused questions.
- A simple, readable dashboard is preferable to a visually overloaded interface.

Chapter 03: SYSTEM DEVELOPMENT

3.1 Design of the Project

The COVID-19 Data Visualization system is designed as a linear analytics pipeline. Raw COVID-19 data is loaded from a CSV file, validated, standardized and cleaned. The prepared data is then transformed for analysis, used to calculate KPIs and passed to the visualization layer. The dashboard layer provides filters, KPI cards, charts, a dataset preview and a download option. Separating data preparation from presentation makes the source code easier to understand and maintain.

COVID-19 Data Visualization – System Architecture

3.2 Functional and Non-Functional Requirements

#	Requirement	Description
FR1	Data Import	Read the COVID-19 CSV dataset from the configured data path.
FR2	Column Detection	Identify common names for date, location and case-related fields.
FR3	Data Cleaning	Convert numeric fields, remove duplicates and handle invalid values.
FR4	Date Processing	Convert valid dates and derive year/month attributes.
FR5	KPI Calculation	Calculate confirmed, death, recovery and active-case summary values when available.
FR6	Filtering	Allow date-range and location selection when supported by the dataset.
FR7	Trend Visualization	Display time-based trends using interactive Plotly line charts.
FR8	Location Comparison	Display top locations by confirmed cases using a bar chart.
FR9	Data Preview	Show a preview of the processed dataset.
FR10	Data Download	Allow the cleaned dataset to be downloaded as CSV.

Non-Functional Requirements

#	Requirement	Description
NFR 1	Performance	The dashboard should respond appropriately for the project-scale dataset.
NFR 2	Usability	Labels, filters and visual elements should be clear to a non-technical user.
NFR 3	Maintainability	Data-processing logic should be separated into reusable functions.
NFR 4	Reliability	KPI calculations should be based on the processed dataset consistently.
NFR 5	Data Integrity	Cleaning operations should preserve the intended meaning of the source fields.
NFR 6	Accessibility	Charts and dashboard elements should remain readable and clearly labeled.

3.3 Technologies Used

Technology	Role in the Project
Python	Primary programming language for data processing and dashboard implementation.
Pandas	Loads, cleans, transforms, filters and aggregates tabular COVID-19 data.
NumPy	Supports numerical data operations and is included in the project environment.
Plotly	Creates interactive line and bar visualizations.
Streamlit	Provides the interactive dashboard interface, filters, KPI cards and download controls.

3.4 Folder Structure

The implementation can be organized so that the source code, dataset, dependencies and documentation remain easy to locate. The structure below matches the dashboard-oriented source-code setup used for the project.

```
COVID-19-Data-Visualization/
■
■■■ data/
■ ■■■ covid19_dataset.csv
■
■■■ app.py
■■■ requirements.txt
■■■ README.md
■
■■■ docs/
■■■ Abstract.pdf
■■■ Algorithm_and_Dataset.pdf
■■■ Source_Code.pdf
```

Folder descriptions:

- **data/** contains the COVID-19 CSV dataset used by the application.
- **app.py** contains the Streamlit dashboard and reusable data-processing functions.
- **requirements.txt** contains the Python dependencies.
- **docs/** contains supporting academic documentation and source-code material.

3.5 Data Model / Analytics Architecture

The project is primarily a descriptive analytics application rather than a transactional system. Therefore, the logical data model focuses on a central COVID-19 fact table and supporting analytical dimensions. The model is conceptual and can be implemented directly through Pandas group-by operations in the current Streamlit application.

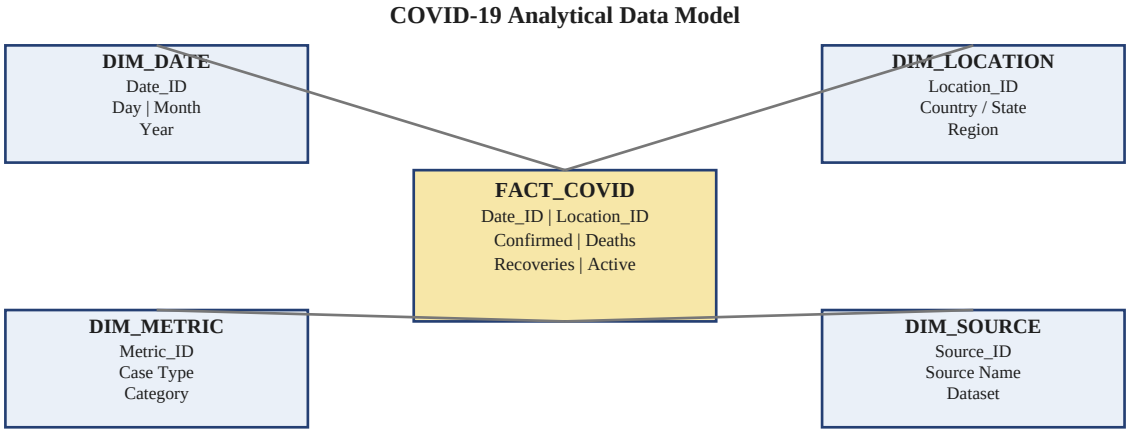


Fig 3: COVID-19 Analytical Data Model

The conceptual FACT_COVID layer contains date, location and numerical COVID-19 measures. DIM_DATE supports time-based analysis, DIM_LOCATION supports regional or country comparisons, DIM_METRIC represents case categories, and DIM_SOURCE identifies the dataset source. This structure provides a clear analytical view even though the current source code processes the CSV directly rather than creating a physical database schema.

3.6 Pseudo Code

```
BEGIN COVID19_Data_Pipeline
dataset_path = "data/covid19_dataset.csv"

IF dataset does not exist:
DISPLAY error message

raw_data = READ_CSV(dataset_path)

STANDARDIZE column names
IDENTIFY date, location and COVID metric columns

IF date column exists:
CONVERT date to datetime
REMOVE invalid dates
CREATE year and month fields

FOR each supported numeric field:
CONVERT values to numeric
REPLACE invalid numeric values with 0

REMOVE duplicate records
PREPARE cleaned dataset

IF date filter is available:
APPLY selected date range

IF location filter is available:
APPLY selected locations

CALCULATE KPI values:
confirmed cases
deaths
recoveries
active cases

DISPLAY KPI cards

IF confirmed cases and date are available:
DRAW confirmed cases trend

IF deaths and date are available:
DRAW deaths trend

IF recoveries and date are available:
DRAW recoveries trend

IF location and confirmed cases are available:
GROUP BY location
DISPLAY top locations by confirmed cases

DISPLAY first 100 rows of processed data
PROVIDE cleaned CSV download

END
```

3.7 Flow of the Application

The end-to-end flow is: COVID-19 dataset → data loading → column standardization → data validation → cleaning → date and numeric transformation → dashboard filters → KPI calculation → Plotly visualization → Streamlit presentation → processed-data download. Each stage uses the output of the previous stage so that the final dashboard is based on a consistent processed DataFrame.

3.8 Dashboard / Visualization Layout

The following layout represents the dashboard components implemented by the project source code. It is a documentation diagram showing the intended arrangement of KPI cards, trend charts, location comparison and dataset preview; it is not a claim of measured COVID-19 results.

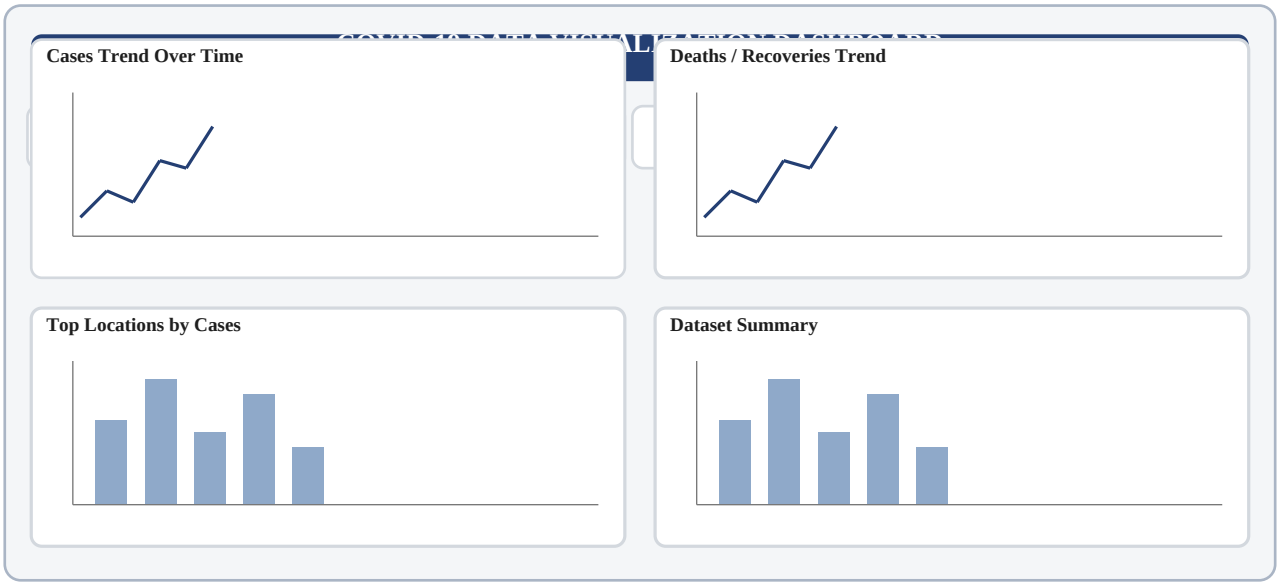


Fig 4: COVID-19 Dashboard / KPI Visualization Layout (Illustrative)

The dashboard presents summary KPI cards at the top, followed by trend and comparison charts. The interface also provides date and location controls and a processed dataset preview. This arrangement allows users to move from a high-level summary to detailed visual analysis without leaving the main dashboard.

Chapter 04: PERFORMANCE ANALYSIS

This chapter evaluates the project from a data-processing, visualization and usability perspective. Because the project is an academic-scale implementation and no formal load-testing study was supplied, the analysis is qualitative rather than a report of execution-time benchmarks or accuracy percentages.

4.1 Data Processing Efficiency

The source code organizes preprocessing into reusable functions rather than scattering transformations across the dashboard. The pipeline performs column standardization, date conversion, numeric conversion, duplicate removal and filtering before the visualization layer. This reduces repeated data-preparation logic and makes the application easier to maintain.

4.2 Dashboard Responsiveness and Interactivity

The Streamlit interface provides interactive controls for date ranges and locations when those fields exist. Changing a filter updates the processed DataFrame used by the KPI and chart sections. Plotly provides interactive chart behavior, allowing users to inspect visual trends without manually creating separate reports.

4.3 Data Consistency and KPI Reliability

KPI calculations are derived from the cleaned and filtered dataset rather than from hard-coded values. The application also uses helper functions to locate available columns and to format numerical values. This approach reduces dependence on one exact dataset naming convention and helps the dashboard remain usable when field names vary slightly.

4.4 Visualization Effectiveness

Line charts are used for time-based trends because their horizontal axis naturally represents chronological progression. Bar charts are used for location comparison because the primary question is categorical comparison. KPI cards provide compact summary values. The combination avoids overloading a single chart with unrelated information.

4.5 Usability and Analytical Value

The dashboard is intended for users who need a quick overview of COVID-19 statistics without directly inspecting raw CSV rows. Filters support focused analysis, while KPI cards and charts provide a layered view of the data. The dataset preview and download feature also make the processed output transparent and reusable.

4.6 Scalability Considerations

The current application is designed for project-scale CSV processing. For substantially larger datasets, performance could be improved by moving data storage to a database, using scheduled preprocessing, caching repeated operations and separating the data service from the dashboard interface. These are future improvements rather than current implementation claims.

Chapter 05: CONCLUSION

The COVID-19 Data Visualization project demonstrates a complete descriptive analytics workflow in which structured COVID-19 records are transformed into an interactive dashboard. The source code performs data loading, column standardization, validation, cleaning, date and numeric transformation, KPI calculation and interactive visualization. The result is a compact interface for exploring COVID-19 indicators across time and available locations.

The project applies practical data-analytics concepts that are relevant to the Computer Science and Engineering (Artificial Intelligence and Machine Learning) curriculum. It shows how data preparation is necessary before meaningful visualization and how interactive dashboards can reduce the effort required to interpret large tabular datasets. The current system deliberately focuses on descriptive analysis and does not present predictive or medical claims.

5.1 Applications

- Educational demonstration of data cleaning and visualization workflows.
- Exploration of confirmed cases, deaths, recoveries and active cases.
- Time-based trend analysis using interactive charts.
- Location-based comparison of reported COVID-19 cases.
- Quick KPI-based summary of the processed dataset.
- Dataset inspection and cleaned-data export for further analysis.
- Foundation for future AIML-based forecasting and anomaly detection.

5.2 Limitations

- The dashboard depends on the quality and completeness of the supplied COVID-19 dataset.
- Exact KPI availability depends on which fields are present in the dataset.
- The current application is based on a CSV workflow and does not provide a live data feed.
- Location comparison requires a suitable location or country field.
- The project does not currently implement machine-learning prediction.
- The dashboard should be treated as an academic analytics tool and not as a medical decision-support system.

5.3 Future Work

- Automated dataset refresh through a reliable API or scheduled data pipeline.
- Real-time or near-real-time data integration.
- Time-series forecasting of COVID-19 indicators.
- Anomaly detection for unusual changes in reported cases.
- Machine Learning models for predictive analysis.
- Additional geographic visualizations such as maps.
- Cloud deployment for wider accessibility.
- Database-backed storage for larger datasets.
- Automated alerts when selected indicators cross user-defined thresholds.

REFERENCES

[1] World Health Organization (WHO), COVID-19 and public-health data resources.

[2] Centers for Disease Control and Prevention (CDC), COVID-19 Data and Surveillance resources.

[3] Pandas Documentation, Python Data Analysis Library, documentation for DataFrame-based data processing.

[4] Plotly Documentation, Interactive graphing library for Python.

[5] Streamlit Documentation, framework for building data and analytics applications in Python.

[6] Python Software Foundation, Python Documentation.

[7] NumPy Documentation, numerical computing tools for Python.

[8] Relevant COVID-19 open datasets used for academic data-analysis and visualization practice.

Note on Sources

The report structure and documentation style are adapted from the uploaded sample project report, which uses sections for abstract, abbreviations, figures, graphs, introduction, literature survey, system development, performance analysis, conclusion and references. The sample also emphasizes data preparation, requirements, technologies, folder structure, analytical architecture, pseudocode, flow and dashboard snapshots. The present report applies that organization to the COVID-19 Data Visualization project while keeping the implementation aligned with the Python/Pandas/Plotly/Streamlit source-code workflow.